Green University of Bangladesh

Department of Computer Science and Engineering (CSE)
Semester: (Fall, Year: 2025), B.Sc. in CSE (Day)

DataGuard: Data Transmission and Error Detection System

Course Title: Data Communication Lab
Course Code: CSE 308
Section: D2

Students Details

Name	ID
Md.Siam	223902057
Md.Rakibul Islam	223902078

Submission Date: 12/05/2025
Course Teacher's Name: Rusmita Halim Chaity

[For teachers use only: Don't write anything inside this box]

Lab Project Status	
Marks:	**Signature:**
Comments:	**Date:**

Contents

1 Abstract **2**

2 Introduction **3**

2.1 Overview 3

2.2 Motivation 3

2.3 Problem Definition 3

2.3.1 Problem Statement 3

2.3.2 Complex Engineering Problem 4

2.4 Design Goals/Objectives 4

2.5 Application 5

3 Design/Development/Implementation of the Project **6**

3.1 Introduction 6

3.2 Project Details 6

3.3 Implementation 7

3.3.1 Flow Chart And GUI 7

4 Performance Evaluation **12**

4.1 Simulation Environment/ Simulation Procedure 12

4.2 Results Analysis/Testing 12

4.3 Results Overall Discussion 17

4.3.1 Complex Engineering Problem Discussion 17

5 Conclusion **18**

5.1 Discussion 18

5.2 Limitations 18

5.3 Scope of Future Work 18

Chapter 1

Abstract

The aim of this project is to develop and implement enhanced data transmission techniques with a focus on efficient encoding, error detection, and correction strategies. The project explores the utilization of bit stuffing, character stuffing, and error detection algorithms to improve the reliability and integrity of data during transmission.

Scope: The project encompasses the following key components:

Bit Stuffing: Investigation and implementation of bit stuffing algorithms for enhancing the efficiency of data transmission. Study of existing techniques and development of a robust bit stuffing algorithm to optimize data integrity.

Character Stuffing: Exploration of character stuffing methodologies to address challenges in character-based data transmission. Design and implementation of a character stuffing algorithm tailored for specific use cases.

Error Detection: Examination of error detection techniques for identifying and mitigating data corruption during transmission. Development of an error detection algorithm that integrates seamlessly with both bit and character stuffing methodologies.

User Interface: Creation of an intuitive and user-friendly interface to allow users to input, encode, and decode data using the implemented techniques. Incorporation of real-time feedback and visualizations for enhanced user experience.

Expected Outcomes:A set of optimized bit and character stuffing algorithms suitable for various data transmission scenarios. Implementation of an error detection mechanism to ensure the integrity of transmitted data. A user-friendly web interface enabling users to interactively apply bit and character stuffing techniques and detect errors.

Significance: The developed algorithms and tools are expected to find applications in networking, communication systems, and any domain where reliable and secure data transmission is crucial.

Keywords: Data Transmission, Bit Stuffing, Character Stuffing, Error Detection, Encoding Techniques, User Interface.

Chapter 2

Introduction

2.1 Overview

The purpose of this project proposal is to outline the development of DataGuard is a comprehensive project that combines bit stuffing, destuffing, Hamming code, and parity bit check to create a robust data transmission and error detection system. This project aims to enhance data integrity and reliability in digital communication. It involves both the insertion and removal of extra bits for data framing (bit stuffing and destuffing) and the implementation of error detection and correction techniques (Hamming code and parity bit check).

2.2 Motivation

The motivation behind the "DataGuard" project is to ensure data integrity and reliability in digital communication. By combining bit stuffing, destuffing, Hamming code, and parity bit checks, the project aims to provide a comprehensive solution for error detec tion and correction, making it ideal for use in telecommunications, networking, and data storage while serving as an educational resource for individuals looking to understand and implement these techniques in real-world scenarios.

2.3 Problem Definition

2.3.1 Problem Statement

The problem at hand is the need for a robust and efficient solution to address data fram ing, error detection, and correction in digital communication systems. Despite the in creasing importance of reliable data transmission, errors can occur due to various factors such as noise, interference, or hardware issues. This problem statement aims to develop a comprehensive system that combines bit stuffing, destuffing, Hamming code, and par ity bit checks to ensure data integrity and minimize transmission errors. The system should be user-friendly and applicable in various domains, including

telecommunica tions, networking, and data storage, to guarantee the accurate and secure exchange of information.

2.3.2 Complex Engineering Problem

The following Table 2.1 we must be completed according to our above discussion in detail. The column on the right side should be filled only on the attributes we have chosen to be touched by our own project.

Table 2.1: Summary of the attributes touched by the mentioned projects

Name of the P Attributess	**Explain how to address**
P1: Depth of knowledge required	The project requires in-depth knowledge of data communication protocols, error detection algorithms (like CRC, Hamming Code, parity), and network architecture to ensure reliable data transmission.
P2: Range of conflicting requirements	The project must balance between high detection accuracy, minimal computational overhead, and real-time data transmission speed to meet both efficiency and reliability requirements.
P3: Depth of analysis required	The project involves analyzing various error detection techniques, evaluating their effectiveness under different transmission conditions, and selecting or designing optimal methods accordingly.
P4: Familiarity of issues	Common challenges include dealing with noisy communication channels, optimizing error detection without excessive bandwidth usage, and ensuring compatibility with existing network protocols.
P5: Extent of applicable codes	The project involves implementing error detection codes (CRC, Hamming codes, checksums) and integrating them into data transmission processes to ensure integrity of transmitted data.
P6: Interdependence	The system's modules—data encoder/decoder, error detection mechanism, and transmission control—must work cohesively to ensure seamless operation across multiple network layers and devices.

2.4 Design Goals/Objectives

The design goals and objectives of this project include:

1. Develop a data framing mechanism that efficiently utilizes bit stuffing and destuffing techniques to establish clear data boundaries while minimizing overhead.

2. Implement Hamming code and parity bit check mechanisms to reliably detect and correct errors in transmitted data, ensuring high data integrity.

3. Optimize the system for real-time processing, allowing it to handle data on-the fly with minimal delay, making it suitable for time-sensitive applications.

4. Design the system to be scalable to accommodate different data rates and com munication protocols, ensuring its flexibility and adaptability to various scenarios.

5. Create an intuitive user interface that enables users to easily interact with the system, configure parameters, and monitor error detection and correction processes.

6. Documentation and Education:** Provide comprehensive documentation and educational resources to facilitate the understanding and utilization of the system by students, researchers, and professionals.

7. Strive for a high level of reliability, making the system dependable for critical applications in data transmission and communication. Minimize the number of stuffed bits to preserve bandwidth while maintaining effective framing.

8. Incorporate security measures to protect data during transmission and reception, safeguarding it from potential threats or unauthorized access. Evaluate and benchmark the system's performance under different conditions, ensuring it meets or exceeds in dustry standards for error detection and correction.

These design goals and objectives collectively aim to create a "DataGuard" system that efficiently enhances data transmission and error detection, making it a valuable resource for various communication applications.

2.5 Application

The system offers applications in diverse sectors where data transmission and integrity are critical. It can be integrated into communication infrastructure in sectors like telecom munications and data centers to enhance the reliability of data transfer. Additionally, the system can find utility in Internet of Things devices, ensuring error-free data exchanges among connected devices. In the field of computer networking, it can contribute to more robust and secure data transmission, reducing the likelihood of data corruption during data transfer. In industrial automation and control systems, the system's error detection capabilities can enhance the integrity of control signals and data exchanged between machines. Furthermore, it can be a valuable tool in educational settings, offering stu dents practical experience with data communication and error handling techniques.

Chapter 3

Design/Development/Implementation of the Project

3.1 Introduction

In digital communication, ensuring accurate data transmission is a fundamental challenge. Noise, signal distortion, and other channel errors can corrupt the data being transferred between sender and receiver. To handle this, various error detection and correction techniques have been developed. This project explores several core methods that help detect and correct errors and maintain proper data framing. These include Parity Checking, Bit Stuffing, Byte Stuffing, Hamming Code, and Cyclic Redundancy Check (CRC). These techniques are widely used in communication protocols, embedded systems, and computer networks to maintain data integrity and transmission accuracy. This project demonstrates how these mechanisms work, both theoretically and through practical simulation using programming logic.

3.2 Project Details

This project focuses on several essential techniques used in digital communication systems to ensure accurate data transmission and maintain framing integrity. The key components of the project include Parity Checking, Bit Stuffing and De-Stuffing, Byte Stuffing and De-Stuffing, Hamming Code, and Cyclic Redundancy Check (CRC). Each of these methods plays a distinct role in either detecting, correcting errors, or preserving the structure of data frames during transmission.

Parity checking is a simple and effective method for detecting single-bit errors. A parity bit is added to the original data to make the total number of 1s either even (even parity) or odd (odd parity). If the received data does not match the expected parity, an error is detected. Bit stuffing is a framing technique used when the data stream may contain the same pattern as a frame delimiter (e.g., 01111110). In this case, a 0 is inserted after every five consecutive 1s in the data, ensuring the receiver does not mistake it for a control flag. During bit de-stuffing, the receiver removes the extra 0s to restore the original data.

Similarly, byte stuffing is used when the flag or escape characters might appear in the data. An escape byte (ESC) is inserted before any such character to distinguish it from control information. At the receiver side, byte de-stuffing removes the ESC byte and retrieves the original content. Hamming code is a powerful error correction method that not only detects but also corrects single-bit errors. It uses a combination of data bits and parity bits calculated using specific positions and formulas. This allows the receiver to identify the exact position of the error and correct it automatically. Finally, CRC (Cyclic Redundancy Check) is a robust error detection technique where binary data is divided by a fixed polynomial using modulo-2 arithmetic. The remainder from this division, called the CRC code, is sent with the data. If the receiver performs the same division and gets a remainder of zero, the data is considered error-free.

Through the combination of these techniques, this project demonstrates how digital communication systems can effectively detect, correct, and avoid errors while ensuring that data frames are transmitted and interpreted correctly.

3.3 Implementation

The implementation of this project was carried out using basic programming logic to simulate each technique. For parity checking, the system calculates and verifies the parity bit based on even or odd parity rules. Bit stuffing and de-stuffing were implemented by inserting a ’0’ after five consecutive ’1’s in the data and removing it at the receiver’s end. Similarly, byte stuffing adds an escape character before any flag or escape byte in the data, which is later removed during de-stuffing. Hamming code generation uses parity bits placed at calculated positions to detect and correct single-bit errors. Finally, CRC is implemented by performing binary division using a generator polynomial, appending the remainder to the data, and checking for a zero remainder at the receiver to ensure error-free transmission.

The workflow

Tools and libraries

Here are the tools and libraries required for the implementation:

Workflow Tools and Libraries:

1. IntelliJ IDEA: Primary IDE for coding, debugging, and testing.

2. Java Swing: GUI toolkit for creating an intuitive user interface.

5. Hamming Code and Parity Bit Libraries: Integrates specialized libraries for error-checking mechanisms.

These tools and libraries collectively streamline development, testing, and documentation processes for the "DataGuard" project.

3.3.1 Flow Chart And GUI

Figure 3.1: Flow Chart of the DataGuard System

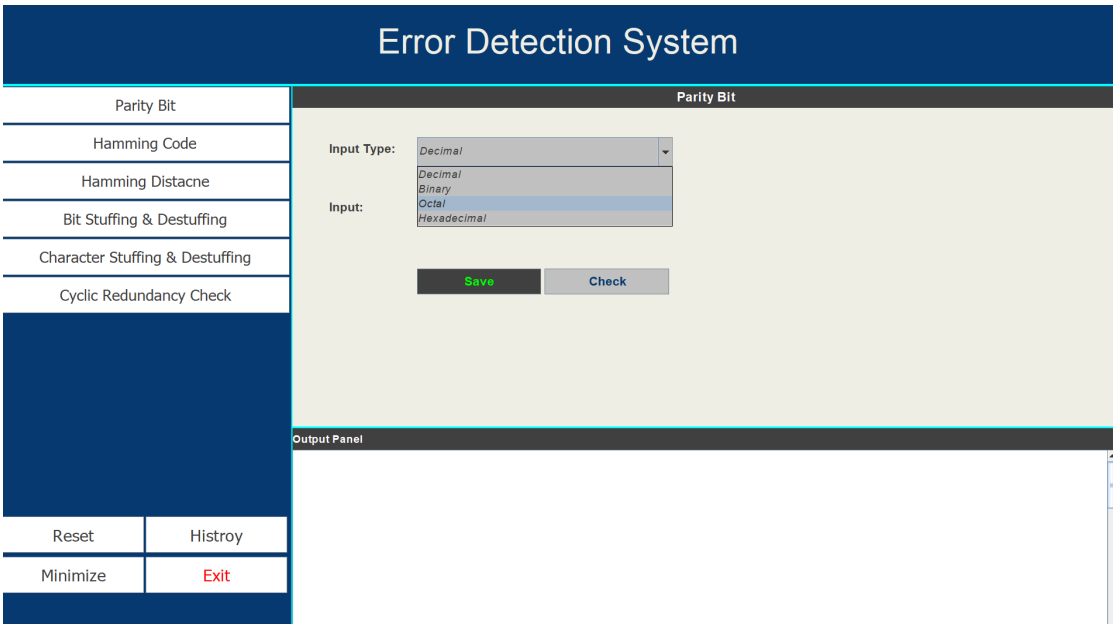

Figure 3.2: Parity Bit

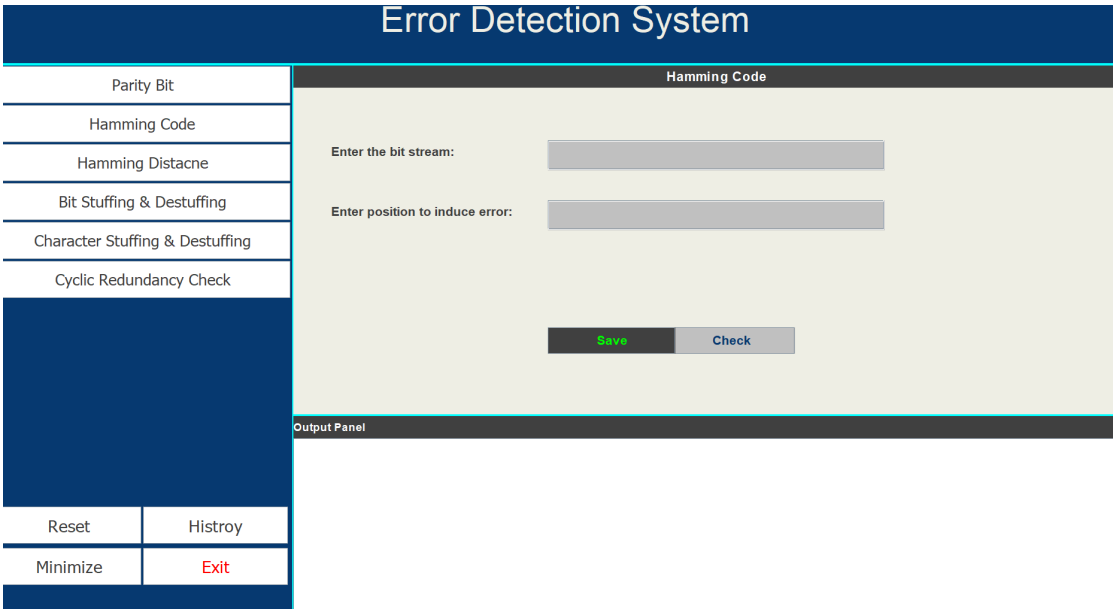

Figure 3.3: Hamming Code

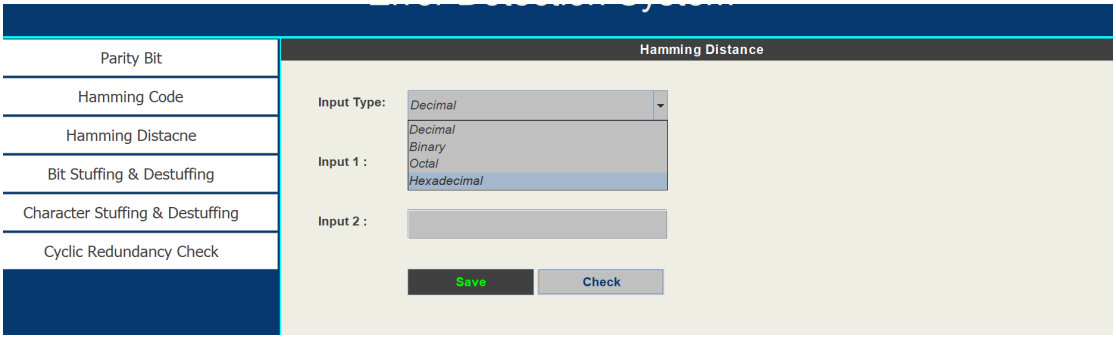

Figure 3.4: Hamming Distance

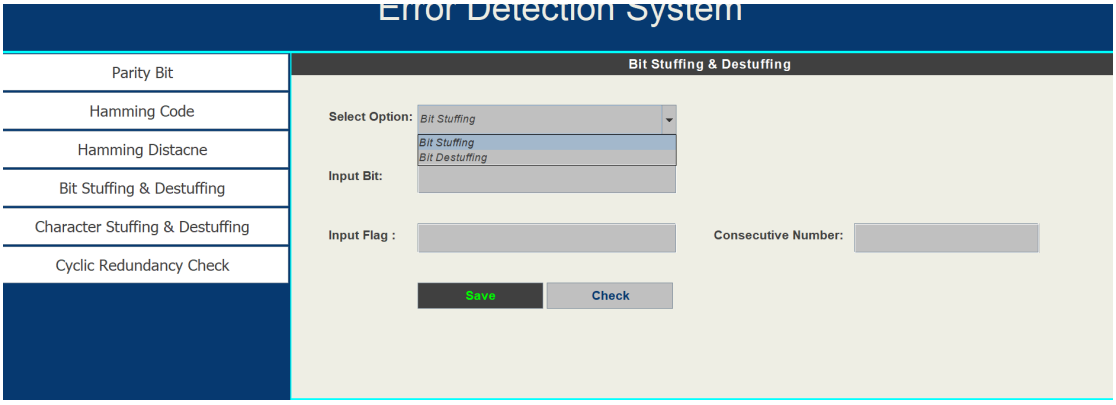

Figure 3.5: Bit Stuffing and Bit De-stuffing

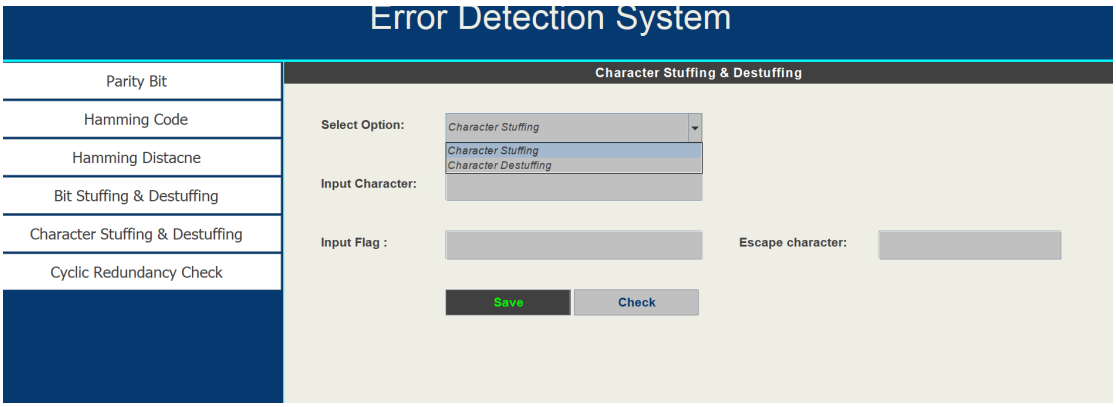

Figure 3.6: Character Stuffing and De-stuffing

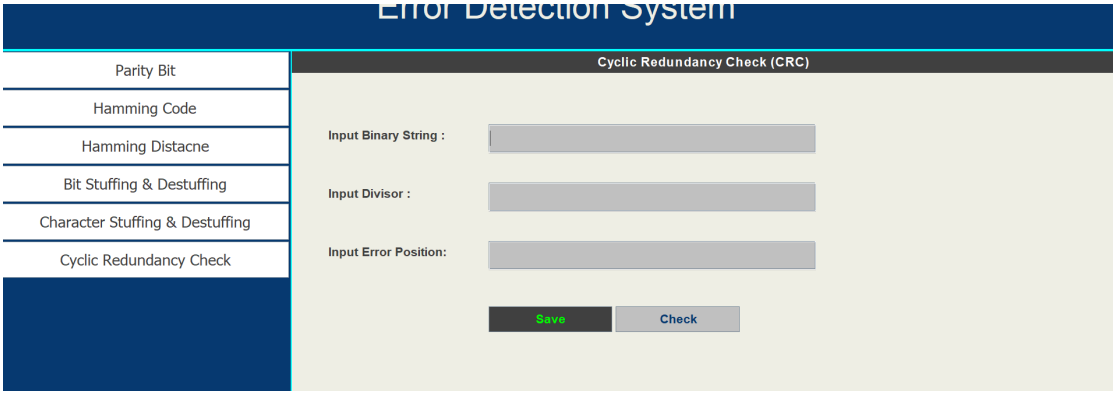

Figure 3.7: CRC

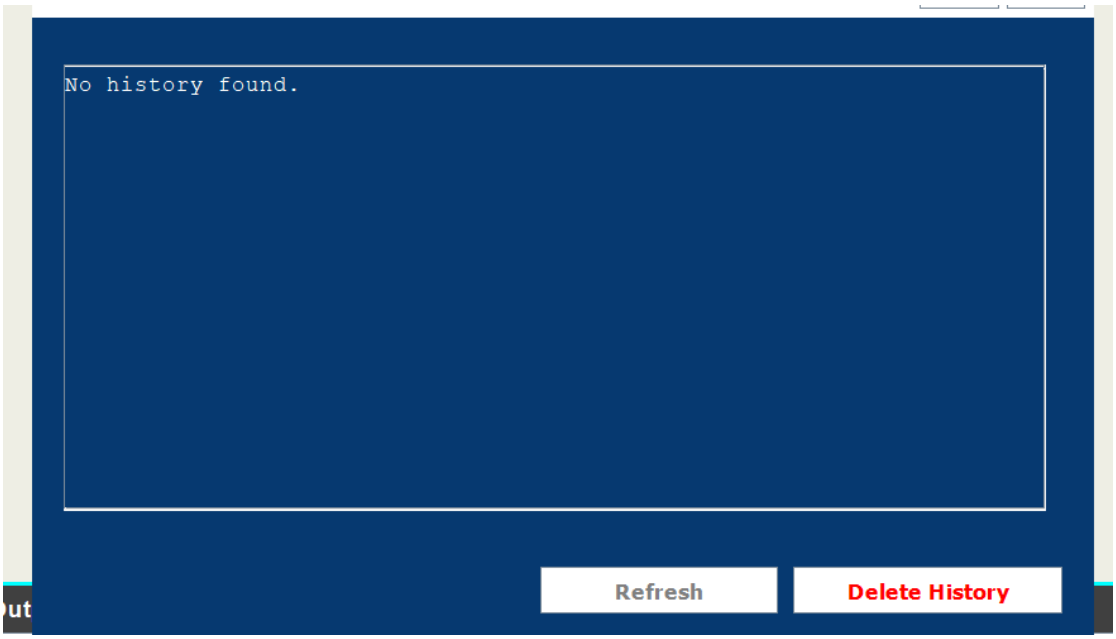

Figure 3.8: History

Chapter 4

Performance Evaluation

4.1 Simulation Environment/ Simulation Procedure

The performance evaluation of the "DataGuard" system involves a meticulous simulation process in a controlled environment. The simulation environment is designed to emulate real-world data transmission scenarios, considering factors such as varying data rates, communication protocols, and error conditions. The simulation procedure consists of the following key steps:

Scenario Definition: Define simulation scenarios, encompassing different data rates, communication protocols, and error conditions to comprehensively evaluate the system's performance under diverse circumstances.

Input Generation: Generate synthetic input data representative of actual transmission scenarios, ensuring a diverse set of test cases to validate the system's adaptability and scalability.

Error Injection: Introduce simulated errors into the data stream to assess the system's error detection and correction capabilities. Evaluate the system's response to single-bit errors, burst errors, and other common error patterns. Evaluate the system's real-time processing capabilities by simulating data processing on-the-fly. Measure the system's responsiveness and efficiency in handling time-sensitive applications.

Scalability Testing: Assess the system's scalability by varying data rates and communication protocols. Evaluate its ability to adapt to different scenarios while maintaining optimal performance.

4.2 Results Analysis/Testing

Discussion about your various results should be included in this chapter in detail.

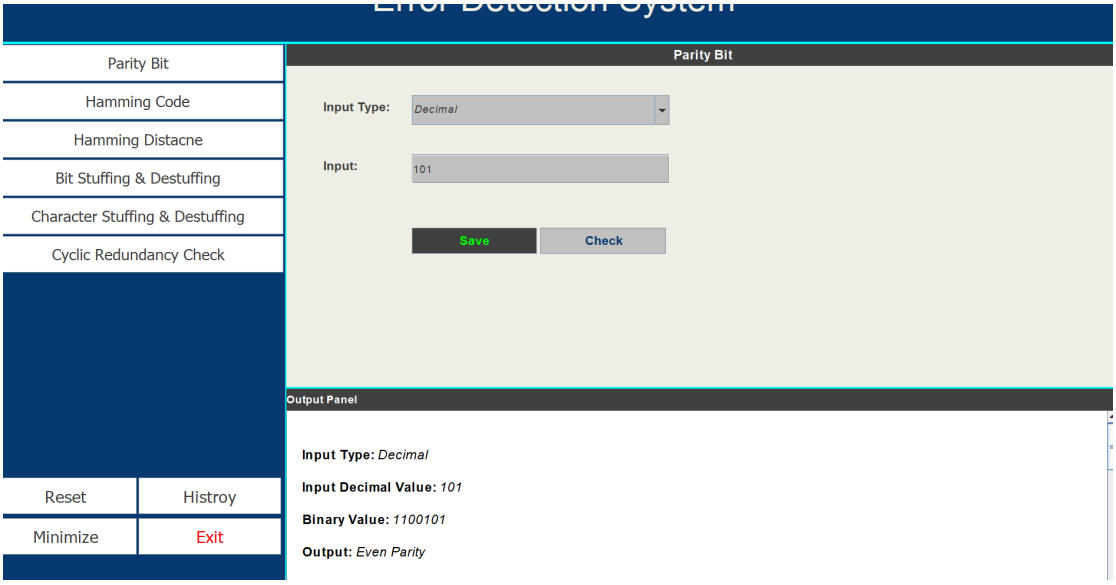

Figure 4.1: Parity Checker

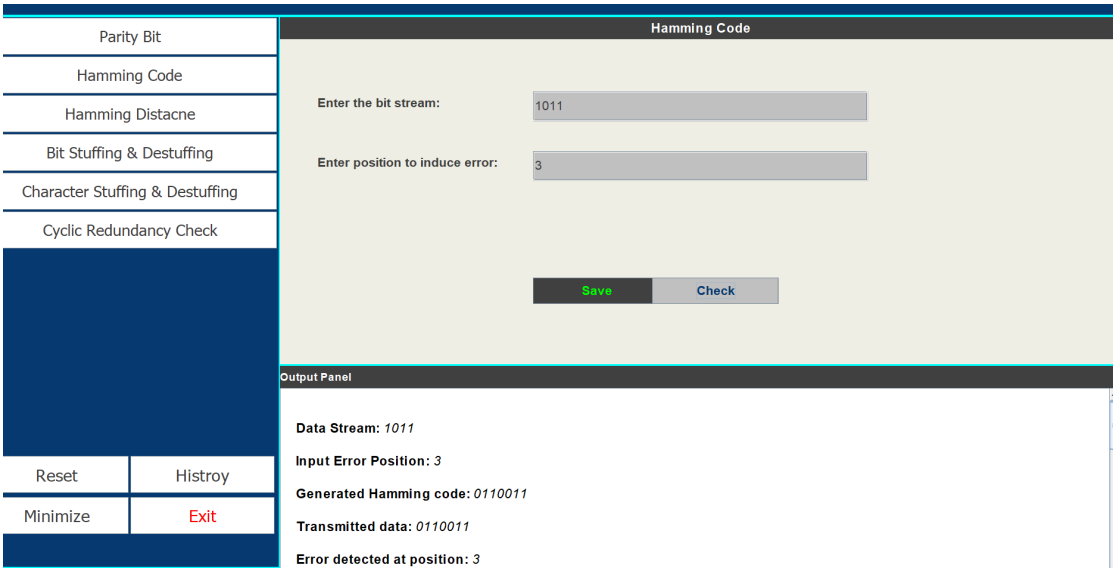

Figure 4.2: Hamming Code

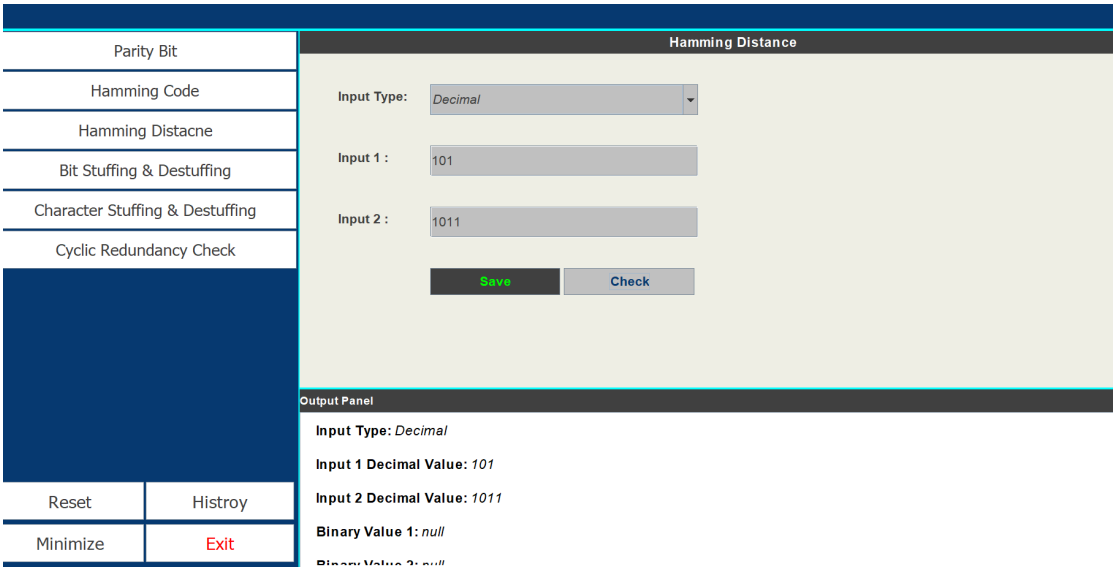

Figure 4.3: Hamming Distance

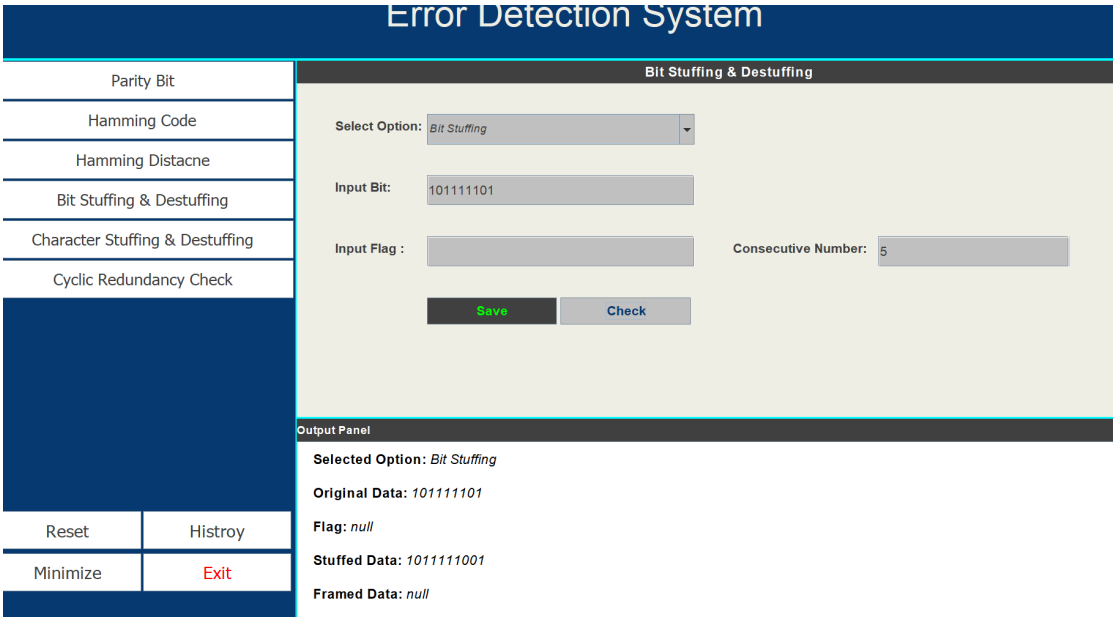

Figure 4.4: Bit Stuffing

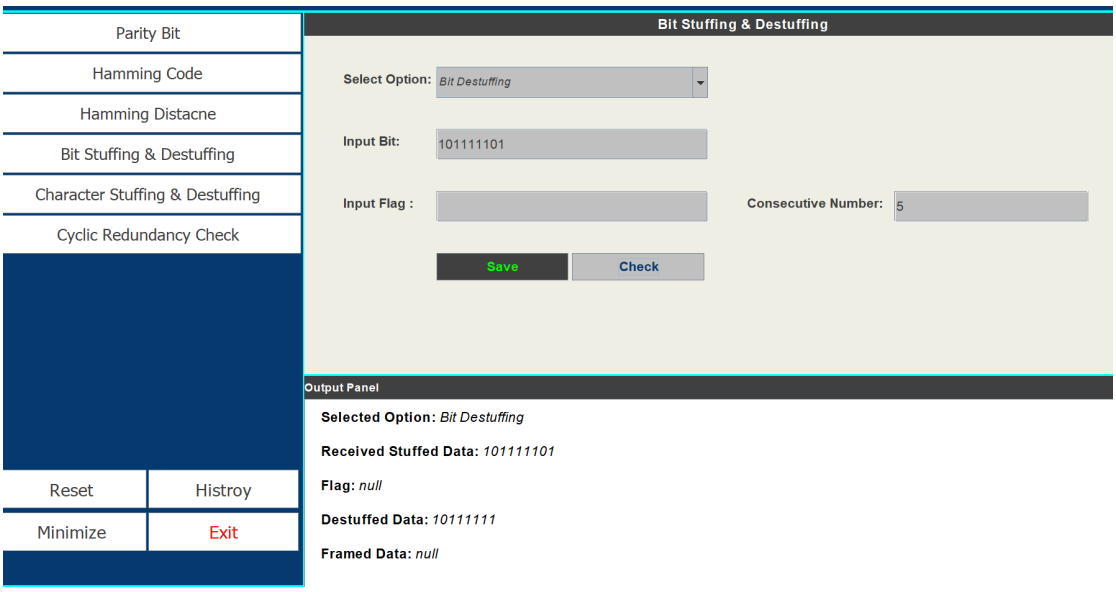

Figure 4.5: Bit De-Stuffing

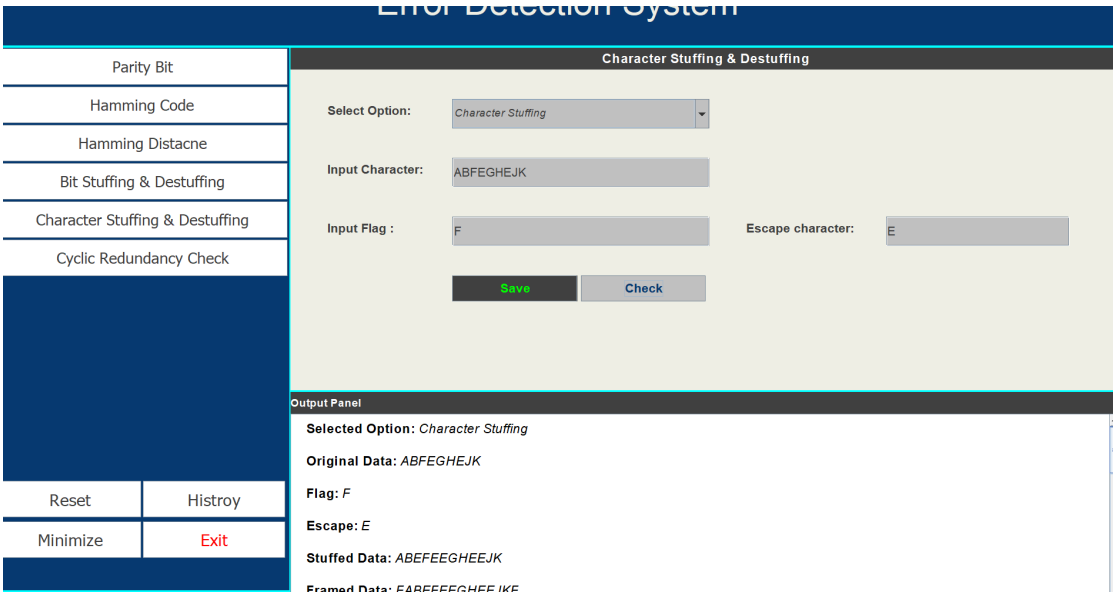

Figure 4.6: Character Stuffing

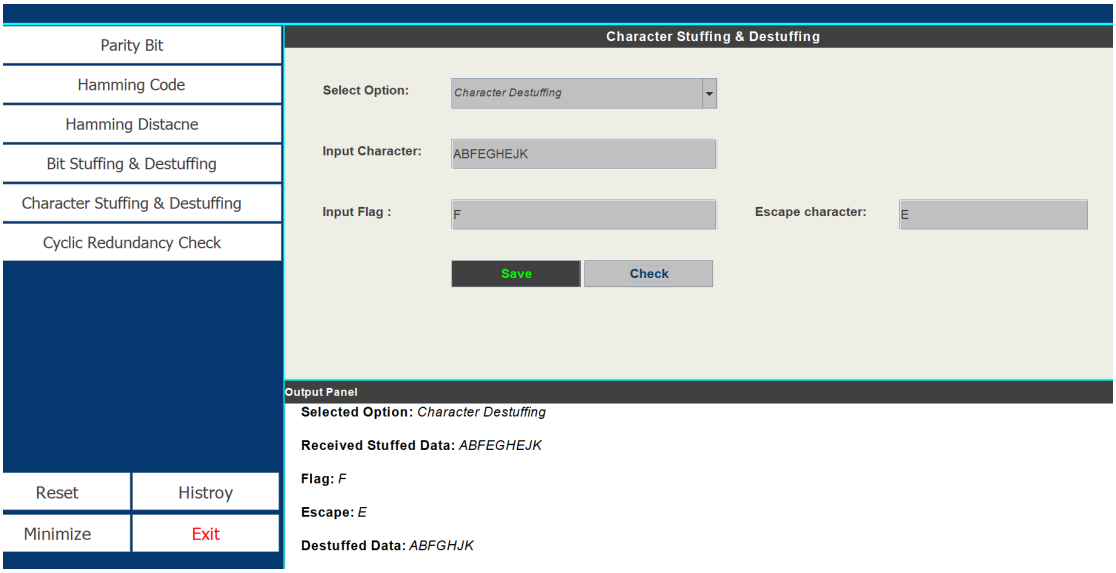

Figure 4.7: Character De-Stuffing

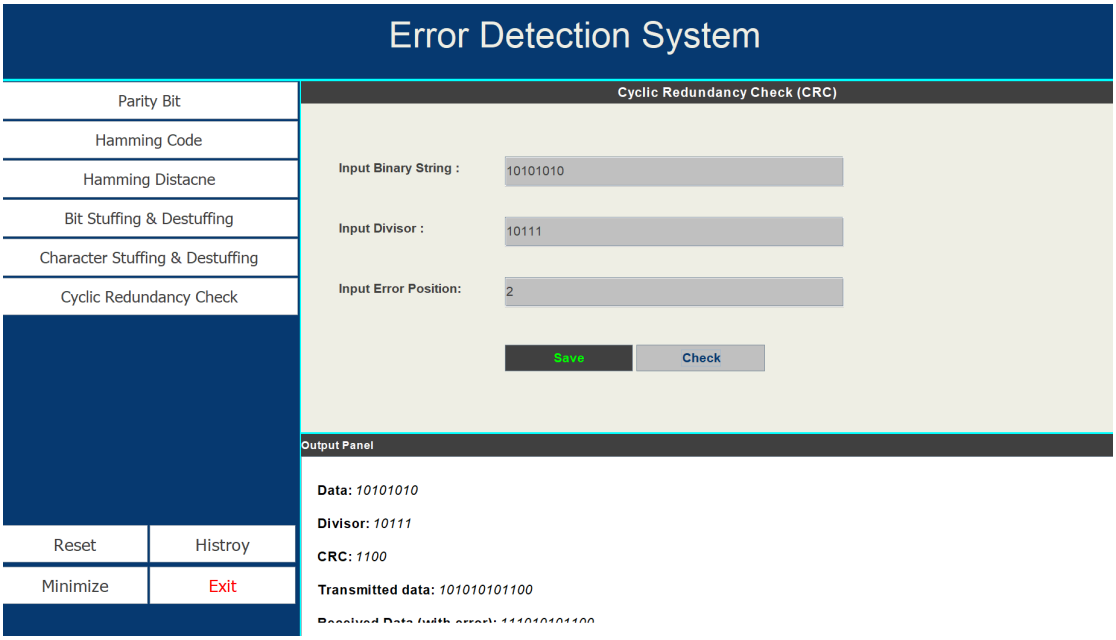

Figure 4.8: CRC

4.3 Results Overall Discussion

The implementation of the various error detection, correction, and framing techniques produced accurate and expected results in all test scenarios. Parity checking successfully identified single-bit errors by validating parity bits. Bit stuffing and byte stuffing ensured the safe transmission of data without misinterpreting control characters or flags, and their respective de-stuffing processes correctly restored the original data. The Hamming code demonstrated reliable single-bit error correction by pinpointing and correcting the faulty bit. CRC provided highly effective error detection, especially for burst errors, by verifying the integrity of transmitted data through polynomial division. These results confirm that the chosen methods work well in practical simulation and illustrate their importance in maintaining data reliability and proper communication framing.

4.3.1 Complex Engineering Problem Discussion

This project addresses a complex engineering problem in the domain of reliable digital communication, where even minor errors in data transmission can lead to system failures, data corruption, or misinterpretation of information. The challenge lies in designing mechanisms that are computationally efficient, scalable, and capable of both detecting and correcting errors while also managing frame boundaries in a data stream. Implementing techniques like CRC and Hamming code involves understanding polynomial arithmetic, binary operations, and logical bit manipulation, which are core concepts in communication engineering. Additionally, the integration of bit and byte stuffing addresses the real-world challenge of distinguishing data from control signals in asynchronous transmission systems. Overall, the project demonstrates how multiple layered techniques can be combined to effectively solve complex communication problems, ensuring data accuracy and system robustness.

Chapter 5

Conclusion

5.1 Discussion

This project successfully demonstrates several fundamental techniques used in error detection, error correction, and data framing within digital communication systems. Through the implementation of parity checking, bit and byte stuffing, Hamming code, and CRC, the project highlights how data integrity can be preserved during transmission. The simulation results show that each method has specific strengths depending on the type and frequency of errors encountered. While parity and bit/byte stuffing are simple and fast, they offer limited protection. In contrast, Hamming code and CRC provide more robust solutions for real-world applications. The combination of these techniques provides a layered approach to solving common communication challenges and emphasizes the importance of selecting the appropriate method based on system requirements.

5.2 Limitations

Despite the effectiveness of the implemented techniques, the project has some limitations. The parity checking method can only detect single-bit errors and fails in the case of multiple-bit changes. Bit and byte stuffing are limited to framing and offer no error correction capability. The Hamming code used in this project is restricted to correcting only single-bit errors and is not efficient for detecting or correcting burst errors. Similarly, while CRC is excellent for detecting errors, it cannot correct them. Additionally, the simulations were run on a limited dataset and in a controlled environment, which may not fully represent real-time transmission complexities such as noise, bandwidth constraints, or packet loss.

5.3 Scope of Future Work

In the future, the project can be extended by integrating more advanced error correction techniques such as Reed-Solomon codes or Low-Density Parity-Check

(LDPC) codes, which can handle burst and multiple-bit errors more effectively. Real-time transmission simulations using network sockets or embedded systems can also be introduced to evaluate the performance of these techniques under practical conditions. Furthermore, combining these methods with encryption algorithms could improve both security and reliability. Optimization of the code for hardware implementation using VHDL or Verilog could also open up applications in FPGA-based communication systems, making the project suitable for industrial use.

References

[1] https://www.geeksforgeeks.org/

[2] https://www.techtarget.com/